draft 2: August 24, 2025

EvoFlock: evolved inverse design of multi-agent motion

Craig Reynolds
unaffiliated researcher
cwr@red3d.com

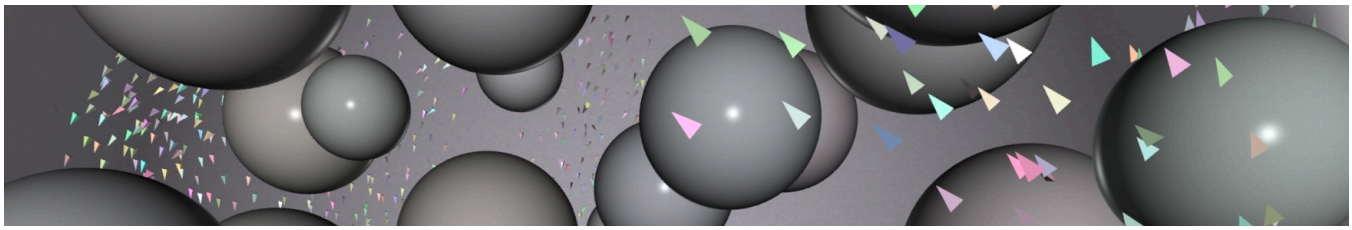

Figure 1: 1000 boids flocking in a space cluttered with obstacles. Behavioral parameters of the boids are determined by inverse design, using multi-objective evolutionary optimization. See this simulation running in Video 1.

Abstract

This paper describes an automatic method for *adjusting* or *tuning* models of multi-agent motion. Simulating the motion of bird flocks, human crowds, vehicle traffic, and other multi-agent systems is a widely used technique. These simulations model the behavior of a single group member (bird, human, or vehicle). The group behaviors (flock, crowd, traffic) *emerge* from interactions between group members. These models typically have many numerical control parameters. Although each parameter may be intuitive in isolation, their interaction can be complex and nonlinear. It is challenging to determine which parameters to adjust for the desired change in group behavior. Changing one aspect of group behavior often causes other aspects to change, leading to a long frustrating process of incremental changes. In this work, the desired group behavior is measured with an objective(/fitness/loss) function and optimized with a genetic algorithm. The objective function used here rewards proper spacing with neighbors, flying near a desired speed, and avoiding obstacles. Interestingly, the vivid alignment seen in bird flocks appears to emerge from maintaining proper spacing between flockmates.

Keywords: multi-agent, inverse design, optimization, evolutionary computation, genetic algorithm, boids, flocks, herds, schools, crowds, traffic

Introduction

Simulation models of multi-agent motion are used in many fields. These include animation, games, biology, robotic swarms, urban planning, training autonomous vehicles, and other applications. Since the 1980s, several models of group motion have been developed, including *boids* and others (see Related Work). These allow creating simulations of flocks and other group motions, which most observers will recognize as some form of flocking. This paper will refer to bird flocks, with the assumption that other types of group motion (herds, schools, crowds, traffic, drone swarms) can be similarly modeled.

This project addresses the issue of *adjusting* or *tuning* multi-agent motion models, toward a given behavioral goal. For example, modifying an existing model of bird flocks to instead portray fish schools. Or starting from a model of flocking crows and changing it to represent flocking sparrows. Similarly, starting from a generic abstract flock model and fitting it to field observations of a particular species of real birds in nature. Or to take a plausibly realistic model of a natural bird flock, and change it, say for storytelling purposes, to convey a flock of birds that are happy, or angry.

A boids-like simulation model typically has a collection of numeric parameters ("knobs") that control its action. *EvoFlock* is a software framework that automatically finds a set of near-optimal parameters for a simulation-based multi-agent system. The flock model used for these experiments is a predefined hand-written "black box" model whose input is a *parameter set* (consisting of 15 numbers, see Table 2). The user provides a *objective* (also known as *fitness* or *loss*) function. It takes a candidate parameter set, runs a simulation, and returns a *score* reflecting how well the behavioral goals were met. The optimization process runs (for about two hours on a laptop) and produces a high quality parameter set, as measured by the objective function.

For a user of this optimization framework, it is convenient that the flock model is treated as a black box. Almost no restrictions are imposed on a preexisting model by the optimization framework. There are no restrictions on model design, source code availability, or the programming language used. (As could be an issue if optimization required language tools such as automatic differentiation (Baydin et al.,

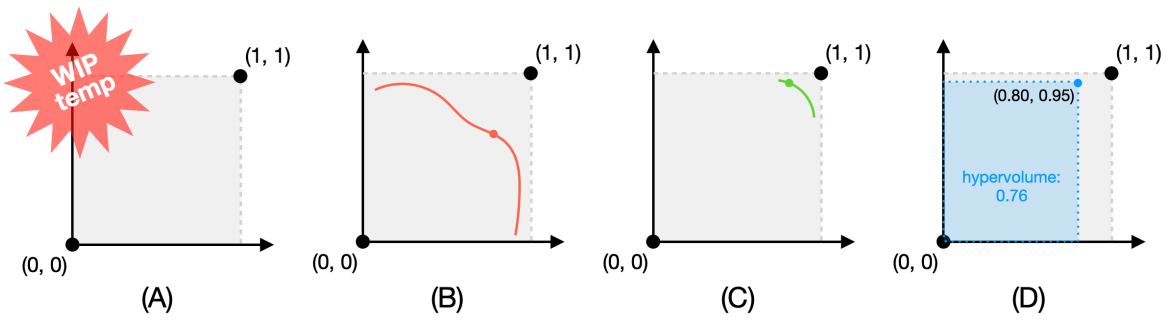

Figure 2: (A) Normalized multi-objective fitness space. For two objectives, a unit square. (B) The *Pareto front* for two mutually conflicting objectives. (Like *reliability* and *affordability* in the hypothetical bridge design example.) The objectives can not all be fully satisfied at any point along the front. Visually, this is the "distance" from (1, 1) to the red curve. (C) Two "mostly compatible" objectives, more typical of criteria for flock tuning. Here, the Pareto front passes near the global optimum. (D) Scalarization of a multiple-objective fitness value, using *hypervolume*: the product of all objective scores. In this 2D example it is the blue rectangle's *area*. Solving a multi-objective problem using scalarized fitness works best for problems of type (C).

2018).)

In the past, flock models have been tuned manually, often a frustrating and time-consuming process. For a new model, adjusting parameters is required simply to create group motion that looks like plausible flocking. Any modification to a group motion model (e.g. birds→fish) requires further adjustments to the parameters. Each such adjustment requires selecting which parameter(s) to change, whether it should increase or decrease, and by how much. Often more than one parameter needs to be changed. The main difficulty is that effects of control parameters may overlap and interact in non-linear ways. Changing one usually requires changing others to compensate. Often, the result is that many parameters need to be changed, leading to a long series of changes and testing.

This paper is about automating that adjustment process using metrics of flock quality and an optimization process. The metrics use multiple objectives. The nature of those objectives and how to add new ones will be described in sections An objective function for flocking and Adding an Objective. The optimization process used in these experiments is a genetic algorithm, described in section Optimization with Genetic Algorithms.

A summary of ("hyper")parameters for this framework is given in Table 1. See anonymized c++ code for this project. See Video 1 of flock simulations based on this approach.

Related Work

Since the 1980s, many simulation-based models of bird flocks have been proposed. They have broadly similar intent, with many differences in their modeling philosophy and computational technique. These models include *boids* and many others: Aoki (1982), Okubo (1986), Reynolds (1987), Heppner and Grenander (1990), Tu and Terzopoulos (1994), Vicsek et al. (1995), Toner and Tu (1998), Couzin et al. (2002), Bajec et al. (2005), Cucker and Smale (2007), Moškon et al. (2007), Cavagna and Giardina (2008), Bajec and Heppner (2009), Bhattacharya and Vicsek (2010) Vásárhelyi et al. (2018) Hoetzlein (2024).

"Tuning" the many parameters of these flock models can be difficult. This led to research on better tools for this task. This paragraph lists previous works that are most similar to EvoFlock. An early, but not very successful, attempt at inverse design for flock simulations (Reynolds, 1993) used *genetic programming* (Koza, 1992). (GP is a variation of the *genetic algorithm* (GA) (Holland, 1975).) Stonedahl and Wilensky (2011) use a GA to search parameter spaces using an objective function running flock simulations based on NetLogo (Tisue and Wilensky, 2004). Similarly Demar and Bajec (2017) attempted "to evolve collective behavior from scratch." Vásárhelyi et al. (2018) used a GA to tune flocking models suitable for the flight dynamics of a swarm of real drones. Stolfi and Danoy (2025) use evolutionary optimization to design robotic swarms for "escorting" tasks.

Most previous work on automatically tuning flock/multi-agent models has used *reinforcement learning* (RL) (Sutton and Barto, 1998) as the optimization technique. For example, see recent excellent work in Cornelisse et al. (2025) and Cusumano-Towner et al. (2025) on making high-quality *driver agents* for a multi-agent traffic simulation. They use *self-play* where agents learn ("bootstrap") from interacting with each other. The technique described in this paper also has an aspect of self-play as flockmates learn to synchronize and cooperate with each other.

As this paper was being drafted, a preprint was posted (Brambati et al., 2025) using reinforcement learning with an objective function very similar to the one used here.

Notable, if somewhat tangential: Jaderberg et al. (2019) uses a hybrid approach to learn human-level game-playing agents for multiplayer games. It uses population-based

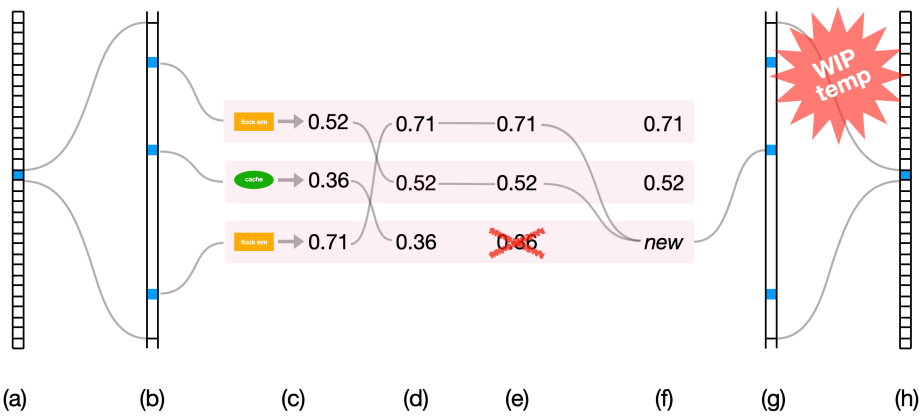

Figure 3: One update step of the (SSGA) evolutionary optimization process, using a 3-way tournament: (a) uniformly select one of the *sub-populations*, (b) from that sub-population, uniformly select three *individuals*, each holding a flock parameter set, (c) get fitness for each, either by running flock simulations (green), or using the fitness cached from previous tournaments (orange), (d) sort the three by fitness, (e) the worst (least fit) individual is deleted, (f) the best two individuals mate (*crossover*) to create a new individual, (g,h) that new individual replaces the worst of the three back in the population. This approach of removing the worst individuals — rather than promoting the best — is a form of *negative selection* which tends to preserve diversity in the population.

reinforcement learning, which combines aspects of RL, population-based GA, and the self-play mentioned above.

EvoFlock was developed as part of a larger ongoing collaboration investigating new approaches to inverse design of multi-agent motion.

Description of the Technique

This section discusses the various components of the optimization process for adjusting parameters of a flock, or other kinds of multi-agent motion models. See Figure 4 for a diagram.

Optimization with Genetic Algorithms

Optimization can be used to find a set of simulation parameters that best fit the behavioral goals, as given by an objective function. EvoFlock uses a *genetic algorithm* (GA), a technique based loosely on concepts of biological evolution, as seen in the natural world. Genetic algorithms were first described by Holland (1975). Evolution was first described by Darwin (1859).

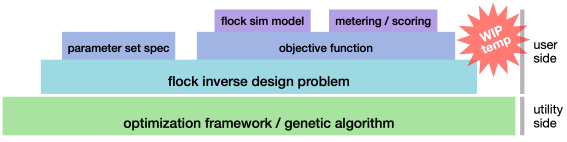

Figure 4: Overall structure of the flock optimization presented here. A generic optimization base (green) operates on a user's flock inverse design problem (blue). The user's description includes a parameter set specification and an objective function, the latter containing a flock simulator and score keeping utilities.

A genetic algorithm maintains a *population* of candidate solutions. Usually, the population contains a fixed number (tens to thousands) of candidate solutions, often called *individuals*. Typically, each individual is a fixed-sized vector of numeric parameter values. The population may be further divided into *demes* or *islands*, representing isolated breeding subpopulations. This is thought to promote diversity by pursuing different solutions in parallel. In an island model, there is usually some kind of *migration* where individuals occasionally move from one island to another.

A GA is a population-based stochastic approach to optimization and discovery. Unlike random search, they are a random process guided by the frequency of useful partial solutions in the population, a model of biological *gene frequency* in a species' *genome*.

The "genome" of a GA changes over time under the influence of the objective function. For example, a model parameter might be changed by *mutation*: adding a small signed zero-centered random value, to change it a bit from its previous value. Perhaps more importantly, individuals are modified by *crossover* where two *parent* individuals are recombined to create a new *offspring*. In a rough model of biological crossover involving parental DNA, two parental parameter sets are merged into one, by taking each parameter at random from one parent or the other (called *uniform crossover*).

Many variations on genetic algorithms have been developed over the years. Sometimes, the entire population of individuals is updated in parallel, as a *generation*, and that

process is repeated tens to hundreds of times. The approach used here, known as *steady state genetic algorithms* (SSGA) updates individuals one at a time (Syswerda, 1991). If a generational GA is run for G generations with a population of P individuals, a corresponding SSGA would be run for $G \times P$ steps. A preexisting GA framework called *LazyPredator* is used for this project. See Figure 3 for a diagram of the SSGA update cycle, and the "tournament of three individuals" based on *negative selection* which promotes diversity in the population.

Objective Function

EvoFlock is based on optimizing a set of parameters for a flock (multi-agent motion) model. Optimization procedures are guided by an *objective function*. In many optimization problems, called multi-objective optimization, the overall objective is a combination of several, potentially contradictory goals.

To motivate this concept, consider building a bridge over a river. A key goal is that the bridge is reliable: that it will carry the required loads without collapsing. Another important goal is to minimize the cost of building the bridge. These criteria are directly opposed. A very strong bridge is reliable but costly. A very cheap bridge is unlikely to be reliable. This trade-off is often called *Pareto optimality*. Tuples of such Pareto optimal values form the *Pareto front*, see Figure 2(b).

Other types of multi-objective optimization have less contradictory goals. The multi-agent motion problem seems to fall into this category. For this application, all that is required is to find parameter sets so that each of the objectives can be fairly well satisfied, see Figure 2(c).

Multi-Objective Optimization

Genetic algorithms were originally used for problems with a single scalar fitness value. These have the advantage of having a *total order*. However, like the bridge building example above, many problems inherently have *multiple objectives* (see Figure 2(B)). The flock optimization problems addressed here seem to be in the "mostly compatible" type of multiple objective problems (see Figure 2(C)) so can be solved with *scalarization*. A *scalarizer* maps from multiple objective values to a single scaler value that somehow summarizes the multiple objective values.

A simple and commonly used scalarizer is called *hypervolume*, which simply multiplies together the components of a multiple-objective value. The name hypervolume refers to a n-dimensional box whose edges correspond to the components of a multiple objective value. Hypervolume has the property that its value changes in proportion to changes in each component.

So, for example, the hypervolume goes up (or down) as any component goes up (or down). This holds as long as none of the components is zero, which "hides" contributions

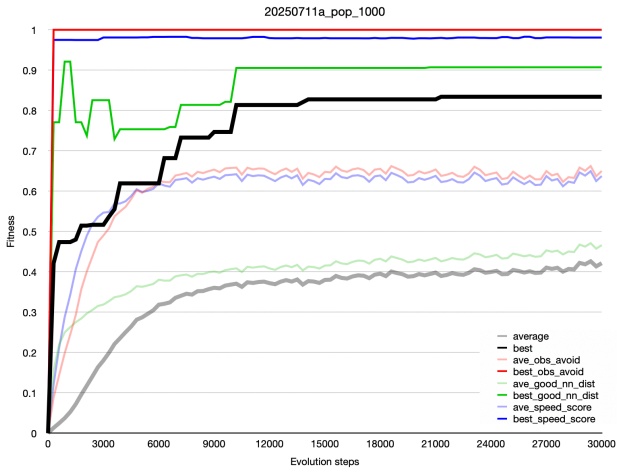

Figure 5: Plot of scalarized fitness (and its component metrics) over 30,000 steps of evolutionary time. The black bold trace (ending at 0.83) is the cumulative best population fitness. The bold gray trace (bottommost) is population average fitness. The colored traces correspond to best and average values (saturated and pastel) for the three component fitness objectives: obstacle avoidance (red), separation (green), and speed (blue). Data from run 20250711a.

of other components. To avoid this, the hypervolume variant used here remaps the fitness range from [0, 1] to [ϵ, 1] before taking the product. (ϵ is 0.01 for the "normalized fitness" values used here.)

Parameter set for a flocking model

The focus of EvoFlock is on *inverse design*, the optimization of parameter sets to achieve group behavior under the direction of an objective function. As such, details of a particular flocking model are largely off-topic. However, just to ground the discussion for readers unfamiliar with flocking models, a quick overview is given here.

Most flock models are written from the perspective of an individual boid (agent) at a single time step. The boid identifies its neighbors. (The model used in this work uses *topological distance* to find the seven nearest neighbors (Cavagna and Giardina, 2008).) The boid reacts to its neighbors by computing *steering forces* related to their distance and angle. It may also need to steer to avoid obstacles. These component steering forces are combined (often by 3D vector addition) then applied to the boid's physics model (often a simple *self-propelled particle* model).

Each of these steering behaviors has parameters, such as weights, distances, and angles. (For example, the weight given to alignment behavior, or the minimum time before a predicted obstacle collision at which the boid should steer to avoid.) There are also parameters related to its physical model, such as the maximum force it can apply.

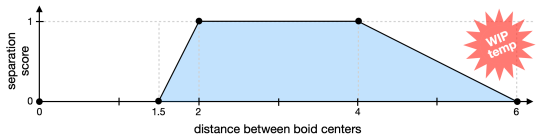

Figure 6: Score for desired *separation* distance to a boid's nearest neighbor. This function is highest when the centers of two boids are between 2 and 4 body diameters apart. The default boid body diameter is 1. The *separation* score for an entire flock simulation is the average of this function, over all *boid-steps*. (That is, for 200 boids on 500 simulation steps, so 100,000 boid steps.)

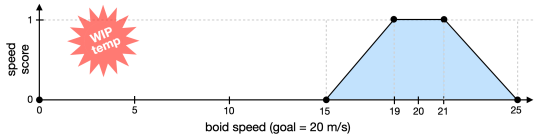

Figure 7: Score for desired boid *speed*, here ∼20 meters per second. This function is highest when speed is between 19 and 21 m/s and falls off to zero outside that range. The *speed* score for an entire flock simulation is the average of this function over all boid-steps.

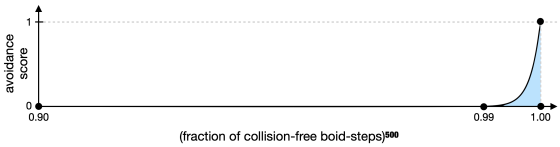

Figure 8: Score for obstacle avoidance. It is the normalized number of collision-free boid-steps, raised to a high (500) power. This function is nearly zero until the number of collision-free steps is about 99%.

The collection of all these make up the flock's *parameter set* which is adjusted by optimization according the the objective function. See Table 2 for a list of the parameter set used in these experiments.

An objective function for flocking

Having described the components of EvoFlock (see Description of the Technique), this section focuses on the specific objective(/fitness/loss) function that was used in this work to optimize a multi-agent motion model for flocking, see Figure 5

The objectives described below are based on measurements made during the flock simulation process. In addition to running the simulation, "bookkeeping" code is run (while updating each individual boid step, after a flock update step, and at the end of a simulation). Those metrics are stored in the flock instance and accessed by the evolutionary optimization framework in which the flock simulation is run.

Objective for Separation

A key observation is that the birds in a flock are grouped closely together but nearly always avoid colliding with flockmates. These goals (close together, no collisions) can be seen as a requirement that the distance between simulated boids fall within a given distance interval. Specifically, for each boid, we determine its nearest neighbor and measure the distance between their centers. A score is computed on the basis of that distance. See Figure 6. It is zero if the distance is too small (potential collision) or too large (not forming a dense flock). Within that acceptable distance range, its value is one. Outside that desired range, piecewise linear ramps transition down to zero.

An equally iconic property of natural flocks is that birds are *aligned*, flying on nearly parallel paths with their neighbors. An interesting result of this work is that this alignment appears to *emerge* from the "acceptable neighbor distance" criteria. It was **not** necessary to have an explicit optimization objective for alignment. The optimization for proper separation appears to **cause** the alignment.

Objective for Obstacle Avoidance

The example application in this work can be described as "flocking in the presence of obstacles" as shown in Figure 1. For collision avoidance, the goal is not to merely *reduce* the number of obstacle collisions, but to effectively eliminate them. These fitness tests (typically) run 200 boids for 500 simulation steps. The relevant statistic is the total number of *avoided* boid-obstacle collisions over 100,000 boid-steps in a flock simulation.

To strongly emphasize the importance of collision avoidance, the normalized obstacle avoidance score (on [0, 1]) is exponentiated to the 500th power. This pushes nearly the whole score's range down to almost zero, except for a small region near a perfect score of one. This effectively rejects flock parameter sets with more than a handful of collisions per flock simulation. An informal threshold in these experiments has been $\leq$10 collisions per 100,000 boid-steps. See Figure 8.

At each time step, part of each boid's simulation is predicting future obstacle collisions and applying steering force

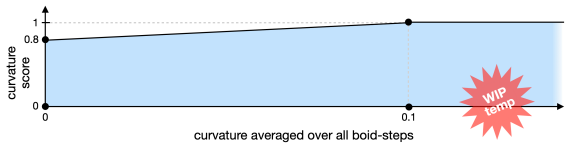

Figure 9: Score for boosting boid path curvature by slightly penalizing especially low curvature. Measured as average over all boid-steps of flock run. Score ramps from 0.8 to 1.0 for curvature on [0, 0.1] then clips at 1 for higher curvature.

to avoid them. The same code detects obstacle avoidance failures. In simulation, this is a matter of a boid having incorrectly passed through the surface of an obstacle (a zero-crossing of the obstacle's signed distance function). A kinematic constraint is invoked to move the boid back to the correct side of the boundary. This failure is recorded in the boid's collision counter.

Objective for Flight Speed

Originally (following early boid models (Reynolds, 1987)) each boid's flight speed was kinematically clipped to remain below 20 meters per second. Later, a third flock optimization objective was added to establish a target speed range. A new behavior was added to each boid to apply a force along its direction of motion to adjust its flight speed toward the desired range. The strength of this behavior became one of the flock parameters being optimized in the inverse flock design process.

Flight speed *can* be left to "float" during optimization. Unfortunately, there is an annoyingly large basin of attractions around not moving (speed=0). It seems likely that any given inverse design problem would imply a range of valid speeds. This suggests that the speed range should be given in the inverse design problem statement. For example, modeling vehicle traffic and pedestrian crowds would imply very different speed ranges. Figure 7 shows a speed score function used in these experiments. It has a target range in the interval [19, 21] with a fixed support ramping down to zero outside that range.

Adding an Objective

EvoFlock aims to simplify modifying flock models. This section describes such a modification, adding a new fitness objective to the "flocking with obstacles" behavior described in the previous section (three objectives: separation in range, speed in range, and avoid obstacles). The flocking motion that results is competent and smooth. Let us assume for some user/application-specific reason, the quality of motion is judged to be "too smooth".

This can be seen as a tendency of the previous three objectives to minimize the boid's *path curvature*. Perhaps intentionally boosting the curvature would counteract that "too smooth" quality of motion? (For example, lending a more "playful" quality of motion?)

The `Boid` class used by EvoFlock already provided an accessor for its instantaneous path curvature. A candidate "flock curvature" score could be simply averaging that over all boid-steps. A test run showed that the instantaneous curvature of a boid's path is approximately on [0, 2]. The average curvature over a run is on [0, 1] at the beginning, but about [0, 0.04] at the end. This supports the notion that basic flocking minimizes curvature.

An objective metric for boosting curvature might map the average curvature of a run to a normalized fitness score. Prototyping this code change took about an hour of programmer effort, followed by a few two-hour optimization runs to find effective "hyperparameters": a linear mapping (with clipping) from a curvature range of [0, 0.1] to a score range of [0.8, 1], see Figure 9. This is effectively a fitness penalty of 20% for no path curvature, ramping up to no penalty for "very curvy."

Description	Value	Notes
boids per flock	200	
flock simulation steps	500	$\sim$16.6 sec
simulation time step	1/30 seconds	
boid body diameter	1 meter	
boid body mass	1 kilogram	
boid max force	< 100 newtons	
boid speed range	19 to 21 m/s	
obs avoid exponent	500	
k nearest neighbors	7	Cavagna (2008)
evolution population	300	individuals
evolution steps	30,000	SSGA steps
equivalent generations	100	
hypervolume epsilon	0.01	ϵ
migrations per step	0.05	
run time: flock sim	$\sim$0.25 seconds	200×500
run time: evolution	$\sim$2 hours	

Table 1: Detailed ("hyper")parameters for the evolutionary inverse design framework. Run times are on a 2021 Apple Macbook Pro M1 Max.

A sample of the resulting flock motion can be seen in Video 2, a simulation with 1000 boids, using flock parameters evolved with this new curvature-boosting objective. (Compare with the non-curvature-boosting Video 1 mentioned in the Introduction.)

Conclusions

This work shows that evolutionary optimization, via a genetic algorithm, can successfully find a set of parameters for a "black box" boid flocking model. This suggests that other similar multi-agent motion models — for example, multi-robot terrain search — might well be solved using a genetic algorithm in a similar way, using an objective with some similarity to the one discussed in section An objective function for flocking.

One advantage of using evolutionary optimization for this type of inverse design problem is that the multi-agent simulation can be treated as a "black box." As such, there are few constraints on how the simulation is implemented. In contrast, using *stochastic gradient descent* (SGD) (Robbins and Monro, 1951) requires that the flock model be differentiable, either analytically or via *automatic differentiation* (Baydin et al., 2018). Conveniently, evolutionary optimization only requires only a flock simulation's scalar fitness value, no

Description	Range	Units
max force	[0, 100]	newtons
forward weight	[0, 100]	*none*
separate weight	[0, 100]	*none*
align weight	[0, 100]	*none*
cohere weight	[0, 100]	*none*
predictive avoid weight	[0, 100]	*none*
static avoid weight	[0, 100]	*none*
max dist separate	[0, 100]	meters
max dist align	[0, 100]	meters
max dist cohere	[0, 100]	meters
angle separate	[-1, +1]	cos(a)
angle align	[-1, +1]	cos(a)
angle cohere	[-1, +1]	cos(a)
fly away max dist	[0, 100]	meters
min time to collide	[0, 10]	seconds

Table 2: Flock model parameters subject to optimization: a collection of scalar values, each with an associated range. They are initially randomized then optimized over time according to the objective function. These values are held in a c++ class called **`FlockParameters`** along with "constant" parameters (flock population, time step, etc.) which are not subject to the optimization process.

gradients are needed.

A surprising finding of this work is that the *alignment* of birds in a flock seems to *emerge* from maintaining proper spacing between flockmates. In retrospect this seems plausible: a group of fast moving animals, with a desire to remain close-but-not-too-close to each other, while dodging around obstacles, must travel on parallel curved paths. However, before these experiments, it was not clear whether an objective function specifically to require alignment would be needed.

A note about GA population size versus the number of SSGA updates (see Figure 3). Some experiments were made increasing the evolutionary population from 500 to 1000. The hope was that more *individuals* (more flock parameter sets) would allow a better solution to be found by using a wider coverage of the parameter space. The opposite seemed to be true. Larger populations led to worse results (often summarized as the best scalar fitness at end of the run). In contrast, reducing the population from 500 to 300 produced somewhat better results, but 150 was worse. Finally, for a population size of 300, more steps seemed to help: running 30,000 updates produced fitness around 0.82 while running 60,000 updates produced fitness around 0.88. Apparently having more updates per individual produced better results.

Limitations

Because EvoFlock is a stochastic algorithm, the quality of the results varies from run to run. (For the three-objective

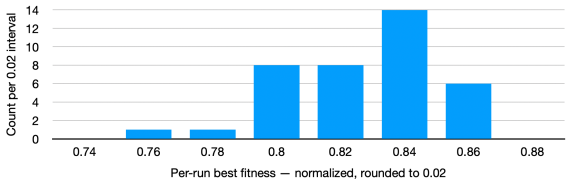

Figure 10: A histogram of final best fitness from 38 identical runs, varying only by random seeds

version, typical end-of-run best fitness is around 0.83, but can be as low as 0.76. See Figure 10.) This procedure is not deterministic, both because the random number generator is seeded from wall clock time for each run and because the objective function is multithreaded and so is inherently nondeterministic.

The current objective function runs **four** flock simulations for each parameter set. These are run in parallel threads on separate CPU cores, so there is little time penalty. However, if not for this 4× load, the cores would be free for other types of parallelism. The four simulations are an attempt to reduce fitness variance of the randomized simulations, to avoid "lucky" high scores. (Of the four fitness values, the *lowest* (worst) is returned from the objective function as a conservative estimate.) Recent work (Antipov and Doerr, 2025) has shown that this may be misguided, and that the quality of results do not suffer from using noisy raw fitness values.

Future Work

All the examples in this work were based on tightly enclosed spaces cluttered with obstacles, see Figure 1. These environments forced the boids to always be avoiding collisions with obstacles or with their flockmates. Other researchers have looked at "open space flocking" (Hoetzlein, 2024) and (Brambati et al., 2025). Parameter sets found with EvoFlock can easily be run in open space but the results are disappointing. This is especially true for the 3 objective version (separation, speed, obstacle), and to a lesser extent, the 4 objective version (with curvature). Digging deeper into open space flocking seems a good direction for future work.

Similarly, multi-agent models with non-uniform agents (Montanari et al., 2025) and non-reciprocal behaviors (Choi et al., 2025), (Weis and Hanai, 2025) are promising future research topics.

The formulation for speed score (Figure 7) and separation score (Figure 6) include "ramps" outside the target range. Are these necessary or even helpful? The robust sampling provided by 100,000 scores per run may tend to "smooth out" those scores without the extra complexity of choosing slopes for ramps.

EvoFlock finds good results using the genetic algorithm (Holland, 1975). It would be interesting to try a version using genetic programming (Koza, 1992). This would elimi-

nate the need to start by hand coding a "black-box flocking model" and could instead evolve it from scratch.

Acknowledgements

This work is part of a larger project. Many thanks for conversations and camaraderie to my collaborators: Gilbert Bernstein, Matthew Shang, and Jennifer Luo. I also want to thank my long time (30 year?) friend and ALife colleague, Inman Harvey, whose deep insights taught me much about evolutionary computation and scholarship. To Andy Kopra for discussions that led to the 1993 paper. Thanks to John Holland for genetic algorithms and to John Koza for introducing me to GP and GA. Thanks to my family for their loving support.

References

Antipov, D. and Doerr, B. (2025). Evolutionary Algorithms Are Significantly More Robust to Noise When They Ignore It. arXiv:2409.00306 [cs].

Aoki, I. (1982). A simulation study of the schooling mechanism in fish. *Bulletin of the Japanese society of scientific fisheries*, 48(8):1081–1088.

Bajec, I. L. and Heppner, F. H. (2009). Organized flight in birds. *Animal Behaviour*, 78(4):777–789.

Bajec, I. L., Zimica, N., and Mraza, M. (2005). Simulating flocks on the wing: the fuzzy approach. *Journal of Theoretical Biology*, 233(2):199–220.

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic Differentiation in Machine Learning: a Survey. *Journal of Machine Learning Research*, 18(153):1–43.

Bhattacharya, K. and Vicsek, T. (2010). Collective decision making in cohesive flocks. *New Journal of Physics*, 12(9):093019. arXiv:1007.4453 [physics].

Brambati, M., Celani, A., Gherardi, M., and Ginelli, F. (2025). Learning to flock in open space by avoiding collisions and staying together. arXiv:2506.15587 [cond-mat].

Cavagna, A. and Giardina, I. (2008). The seventh starling. *Significance*, 5(2):62–66. https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1740-9713.2008.00288.x.

Choi, J., Noh, J. D., and Rieger, H. (2025). Flocking with random non-reciprocal interactions. arXiv:2506.22060 [cond-mat].

Cornelisse, D., Pandya, A., Joseph, K., Suárez, J., and Vinitsky, E. (2025). Building reliable sim driving agents by scaling self-play. arXiv:2502.14706 [cs].

Couzin, I. D., Krause, J., James, R., Ruxton, G. D., and Franks, N. R. (2002). Collective Memory and Spatial Sorting in Animal Groups. *Journal of Theoretical Biology*, 218(1):1–11.

Cucker, F. and Smale, S. (2007). Emergent Behavior in Flocks. *IEEE Transactions on Automatic Control*, 52(5):852–862. Conference Name: IEEE Transactions on Automatic Control.

Cusumano-Towner, M., Hafner, D., Hertzberg, A., Huval, B., Petrenko, A., Vinitsky, E., Wijmans, E., Killian, T., Bowers, S., Sener, O., Krähenbühl, P., and Koltun, V. (2025). Robust Autonomy Emerges from Self-Play. arXiv:2502.03349 [cs].

Darwin, C. (1859). *On the Origin of Species by Means of Natural Selection*. Murray, London.

Demar, J. and Bajec, I. L. (2017). Evolution of Collective Behaviour in an Artificial World Using Linguistic Fuzzy Rule-Based Systems. *PLOS ONE*, 12(1):e0168876. Publisher: Public Library of Science.

Heppner, F. and Grenander, U. (1990). A Stochastic Nonlinear Model for Coordinate Bird Flocks. In *The Ubiquity of Chaos*, pages 233–238. AAAS Publications.

Hoetzlein, R. C. (2024). Flock2: A model for orientation-based social flocking. *Journal of Theoretical Biology*, 593:111880.

Holland, J. H. (1975). *Adaptation in natural and artificial systems*. Complex Adaptive Systems. MIT Press, A Bradford Book, Cambridge, MA. (https://doi.org/10.7551/mitpress/1090.001.0001, 1992 open access edition).

Jaderberg, M., Czarnecki, W. M., Dunning, I., Marris, L., Lever, G., Castaneda, A. G., Beattie, C., Rabinowitz, N. C., Morcos, A. S., Ruderman, A., Sonnerat, N., Green, T., Deason, L., Leibo, J. Z., Silver, D., Hassabis, D., Kavukcuoglu, K., and Graepel, T. (2019). Human-level performance in first-person multiplayer games with population-based deep reinforcement learning. *Science*, 364(6443):859–865. arXiv:1807.01281 [cs, stat].

Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection (Complex Adaptive Systems)*. MIT Press, A Bradford Book, Cambridge, Mass., First edition.

Montanari, A. N., Barioni, A. E. D., Duan, C., and Motter, A. E. (2025). Optimal flock formation induced by agent heterogeneity. arXiv:2504.12297 [cond-mat].

Moškon, M., Heppner, F., Mraz, M., Zimic, N., and Lebar Bajec, I. (2007). Fuzzy model of bird flock foraging behavior. In *Proceedings of EUROSIM 2007*, volume 2, Ljubljana, Slovenia.

Okubo, A. (1986). Dynamical aspects of animal grouping: Swarms, schools, flocks, and herds. *Advances in Biophysics*, 22:1–94.

Reynolds, C. W. (1987). Flocks, Herds and Schools: A Distributed Behavioral Model. In Stone, M. C., editor, *Proceedings of the 14th annual conference on Computer graphics and interactive techniques*, volume 21 of *SIGGRAPH '87*, pages 25–34, New York, NY. ACM.

Reynolds, C. W. (1993). An Evolved, Vision-Based Behavioral Model of Coordinated Group Motion. In *From Animals to Animats 2: Proceedings of the Second International Conference on Simulation of Adaptive Behavior*. The MIT Press. https://direct.mit.edu/book/chapter-pdf/2314157/9780262287159_cby.pdf.

Robbins, H. and Monro, S. (1951). A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22(3):400–407. Publisher: Institute of Mathematical Statistics.

Stolfi, D. and Danoy, G. (2025). Escorting drone swarm formation: a swarm intelligence and evolutionary optimisation approach. *Swarm Intelligence*, 19(3):245–272.

Stonedahl, F. and Wilensky, U. (2011). Finding Forms of Flocking: Evolutionary Search in ABM Parameter-Spaces. In Bosse, T., Geller, A., and Jonker, C. M., editors, *Multi-Agent-Based Simulation XI*, pages 61–75, Berlin, Heidelberg. Springer.

Sutton, R. S. and Barto, A. G. (1998). *Reinforcement Learning: An Introduction*. MIT Press.

Syswerda, G. (1991). A Study of Reproduction in Generational and Steady-State Genetic Algorithms. In Rawlins, G. J. E., editor, *Foundations of Genetic Algorithms*, volume 1, pages 94–101. Elsevier, Amsterdam.

Tisue, S. and Wilensky, U. (2004). NetLogo: Design and implementation of a multi-agent modeling environment. In *Proceedings of Agent 2004*, volume 2004, pages 7–9, Chicago, IL.

Toner, J. and Tu, Y. (1998). Flocks, herds, and schools: A quantitative theory of flocking. *Phys. Rev. E*, 58(4):4828–4858.

Tu, X. and Terzopoulos, D. (1994). Artificial fishes: physics, locomotion, perception, behavior. In *Proceedings of the 21st annual conference on Computer graphics and interactive techniques*, SIGGRAPH '94, pages 43–50, New York, NY, USA. Association for Computing Machinery.

Vicsek, T., Czirók, A., Ben-Jacob, E., Cohen, I., and Shochet, O. (1995). Novel Type of Phase Transition in a System of Self-Driven Particles. *Physical Review Letters*, 75(6):1226–1229. Publisher: American Physical Society.

Vásárhelyi, G., Cs, Somorjai, G., Nepusz, T., Eiben, A. E., and Vicsek, T. (2018). Optimized flocking of autonomous drones in confined environments. *Science Robotics*, 3(20).

Weis, C. and Hanai, R. (2025). Generalized non-reciprocal phase transitions in multipopulation systems. arXiv:2507.16763 [cond-mat].